

AIMer전자서명 AVX512 최적화

일시: 2026년 6월

유튜브: <https://youtu.be/sdDKIUcYePk>

발표자: 이승원

0. 프로젝트 구현 목표

- 2026년 1월 30일 양자내성암호연구단에서 AIMer 표준 문서 및 코드 공개
- Reference Code로 Github에 공개된 코드는 Optimized Code가 제외된 C 코드만 공개
- 이전에 공개된 Reference 코드에서는 Opt 코드가 함께 공개되었고, 해당 코드를 활용하여 KpqClean 통합에 활용
 - 이때 opt 코드는 avx2를 활용한 최적화 구현
- **본 프로젝트는 최신 프로세서에서 제공하는 AVX512를 활용한 AIMer의 최적화 구현을 목표로 함**

The AIMer Signature Scheme	
Authors	Jihoon Cho (Samsung SDS) Jincheol Ha (Cryptolab) Seongkwang Kim (Samsung SDS) Jihoon Kwon (Samsung SDS) Byeonghak Lee (Samsung SDS) Joohee Lee (Sungshin Women's University) Jooyoung Lee (KAIST) Sangyub Lee (Samsung SDS) Dukjae Moon (Samsung SDS) Mincheol Son (KAIST) Hyojin Yoon (Samsung SDS)
Contact	team@aimer-signature.org
Homepage	https://aimer-signature.org
Reference Code	https://github.com/samsungeds-opensource/AIMer Integrity Hash (SHA-256): d7bbdb28 e70dad02 1da2057b c0b48025 85770cbc 2d7449ac fcd770d8 bc624521
Friday 30 th January, 2026	



1. Open Quantum Safe 라이브러리

Open Quantum Safe 라이브러리(liboqs)

- 양자내성암호(PQC) 알고리즘을 위한 오픈소스 C 라이브러리
- KEM(키 캡슐화)·전자서명 알고리즘 구현체를 모아 제공하고, **알고리즘 간 전환이 쉬운 공통 API 지원**
 - NIST 표준 ML-KEM(FIPS 203)·ML-DSA(FIPS 204)와 Falcon, SPHINCS+ 등 표준화 진행 알고리즘 포함
- 테스트·벤치마킹 기능을 내장해 알고리즘 성능 비교·평가에 활용
 - 다양한 플랫폼(Linux·macOS·Windows, x86_64·ARM)과 언어 래퍼
 - OpenSSL/OpenSSH 통합 제공
- 공통 API와 표준화 기반 구조를 갖춰 AIMer 최적화 구현에 활용 가능하며, 구현을 liboqs 구조에 맞춰 통합해 두면 **향후 OQS에 합치기에도 용이**



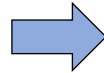
1. Open Quantum Safe 라이브러리

KpqClean_Ver2 · PQClean 방식 (변종마다 자기완결)

```
crypto_sign/  
├─ AlMer128f/ {clean, avx2}/  
│   api.h sign.c hash.c field.c  
│   aim2.c tree.c params.h  
│   Makefile PQCsignKAT... (17개)  
├─ AlMer128s/ ... 192f/s ... 256f/s  
│   ↑ 6변종 × 2(clean, avx2) = 전부 복제  
common/ fips202-aes-rng (일부만 공유)
```

문제

hash·field·Makefile·KAT 중복 /
호출 API가 변종별로 다름(crypto_sign_*) /
clean·avx2 폴더·빌드 분리



OQS (liboqs) 구조

```
include/oqs/ ← 공개 헤더 단일 트리  
    sig.h common.h sha3.h ...  
src/common/ ← 공통 코드 1벌만  
    sha3(Keccak) sha2 aes rand  
src/sig/aimer/  
    sig_aimer_128f.c ← 통합 래퍼  
    aimer-128f_{ref, avx2, avx512}/  
tests/KAT/ ← 중앙집중
```

이점

통합 API OQS_SIG /
common 1벌 공유 /
런타임 ref·avx2·avx512 디스패치

- 왜 common을 OQS로?

① common 단일화

Keccak/SHA/AES/rand를 한 곳에.
AVX512 Keccak을 common에 한 번만 넣으면 6변종 전부 즉시 가속 — 중복 제거.
유지보수 1지점

② 통합 API (OQS_SIG)

변종을 이름으로 고르는 단일 인터페이스 (함수포인터+길이 메타). PQClean식 변종별 crypto_sign_* 분기 제거
→ 벤치·연동 일관

③ 런타임 디스패치

래퍼 1개가 CPU 기능에 맞춰 ref/avx2/avx512 선택.
한 바이너리로 이식성과 최적화를 동시에

④ 표준 생태계

liboqs = 사실상 PQC 통합 표준.
그 구조를 따르면 외부 비교·제출·통합이 쉬움



2. AIMer 표준화 변경 사항

KpqClean Ver2

- 2024년 7월 17일 공개
- KpqC 공모전 2차 공모를 진행하는 알고리즘의 벤치마크 라이브러리

2025. 01. 16 최종 선정 알고리즘 공개

2026. 01. 30 표준 문서 및 코드 공개

AIMer 코드 변경

KpqClean ↔ AIMer 표준 코드 간 **KAT 불일치**

FIPS 204(ML-DSA)와 동일하게
Sign 메시지에 context 추가

– **Sign**(sk, m) $\rightarrow \sigma$: Take as input a private key $sk = (pt, iv, ct)$ and a message $m \in \{0, 1\}^*$, and compute the zero-knowledge proof π for the AIM2 one-way function circuit using m as a part of the input to the challenge hash as described in Algorithm 8. Output the corresponding signature $\sigma \leftarrow \pi$ where $|\sigma| = (5 + (\log_2 N + \ell + 5)\tau)\lambda$

AIMer.sign(sk, M, ctx) $\rightarrow \sigma$: Take as input a private key $sk = (pt, iv, ct)$, a message $m \in \{0, 1\}^*$, and a context string ctx , and compute the zero-knowledge proof π for the AIM2 one-way function circuit using m as a part of the input to the challenge hash as described in Algorithm 10. Output the corresponding signature $\sigma \leftarrow \pi$ where $|\sigma| = (5 + (\log_2 N + \ell + 5)\tau)\lambda$

2.2.2. Context Strings

ML-DSA supports an optional **context string** (ctx) of up to 255 bytes. The **context string** is incorporated into the signed message and provides domain separation between different applications or uses of the same key pair.

If a **context string** is used during signing, the same **context string** is required during verification. Defining a fixed **context string** for a given protocol's use case prevents cross-protocol attacks.

3. AIMer AVX2 적용

- Kpqclean Ver2 – AIMer_AVX2 구현을 가져와서 그대로 사용
 - 표준 코드에서는 AVX2 최적화 구현 미제공
 - AVX2 구현은 Ref 대비 **3~4배 성능 향상**(Kpqclean 기준)

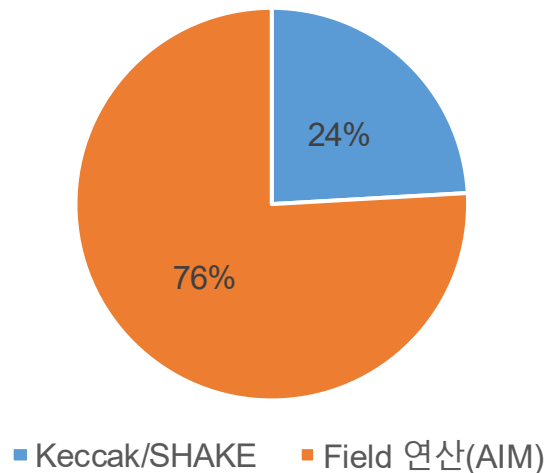
구성 요소	Clean	AVX2
Keccak/SHAKE	단일 Shake 128 연산	4-way 병렬 Keccak 4개의 Shake를 병렬 연산
GF 곱셈	스칼라 carry-less 곱셈	PCLMULQDQ 하드웨어 가속기 명령어 활용
Field 연산 단위	단일 원소 (GF_mul_s, GF_sqr_s)	N-party 배치 (GF_mul_n, GF_sqr_n..) N개 파티 동시 처리

1차 작업 : Reference Code → LIBOQS 구조 변경 → **AVX2 적용(Baseline)**



4. AIMer 최적화 포인트

AIMer128f-ref 연산 비중

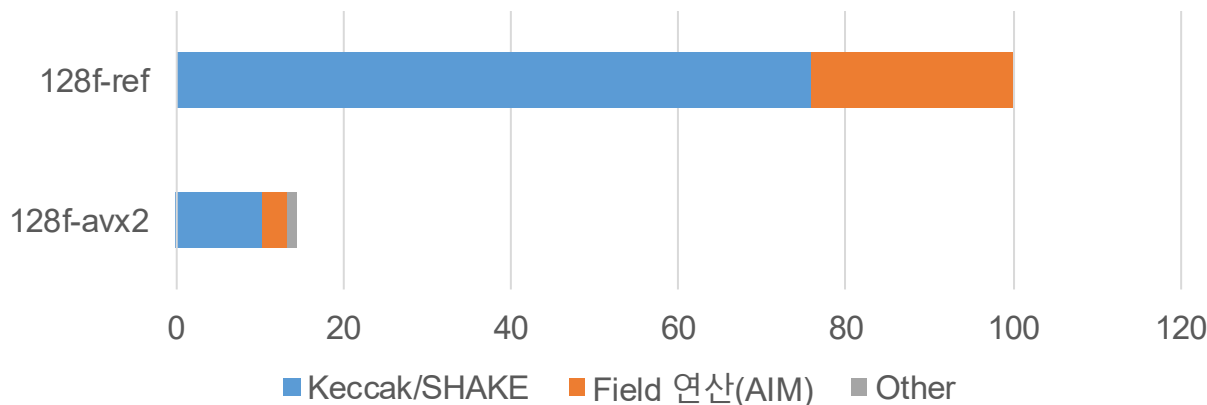
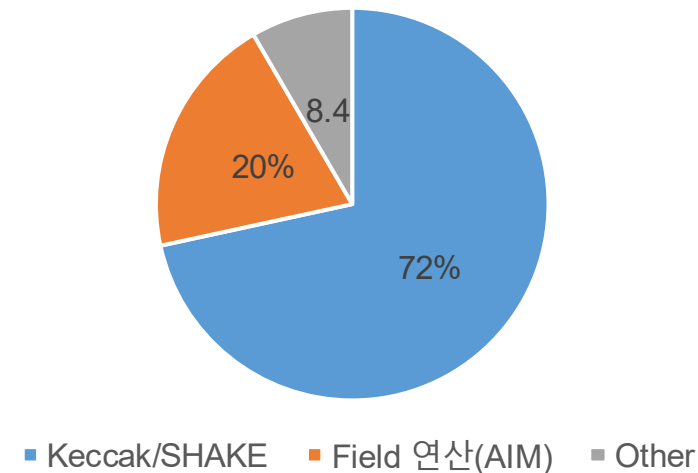


Ref → AVX2

Field 연산 비중 감소
76% → 20%

Keccak/SHAKE 연산 비중 증가
24% → 72%

AIMer128f-avx2 연산 비중



절대 성능 기준으로 비교하면
Keccak/SHAKE의 성능도 크게 향상

프로파일링 분석 결과만으로 판단하였을 때,
AVX512 적용시 Field 연산 대비

Keccak/SHAKE 최적화 효율이 높을 수 있음

4. AIMer 최적화 포인트

- Intel AVX-512 기반 Keccak 최적화(25. 10. 27)
 - 512비트 벡터 레지스터와 **VPTERNLOGQ** 명령어를 활용
 - State 64비트를 512비트 레지스터에 병렬 적재
 - 비트 단위 논리 연산인 **Chi 단계**를 VPTERNLOGQ 명령어를 통해 1 cycle에 단일 처리 하여 성능 향상
- 실제로 ML-KEM/ML-DSA의 성능이 AVX2 대비 **최대 1.6배 향상**
- OQS 라이브러리에 적용되어 동작 및 활용 가능

Release notes

This is version 0.14.0 of liboqs. It was released on July 10, 2025.

This release contains a security fix for secret-dependent branching in HQC. It introduces support for SNOVA, a NIST Additional Signatures Round 2 candidate, and **a new optimized implementation of SHA3 using AVX-512VL instructions**. Additionally, this is the first liboqs release to include the **stable 1.0.0 version of PQ Code Package's mlkem-native**.

This release also introduces a number of improvements to testing and infrastructure. The OQS project is now publishing benchmarking data on <https://openquantumsafe.org/benchmarking> and code coverage data on <https://coveralls.io/github/open-quantum-safe/liboqs>.

White Paper

intel

Accelerate Post-Quantum Cryptography with Intel Crypto Technologies

Intel supports the transition to Post-Quantum Cryptography (PQC) through a range of quantum resilient platforms that deliver best-in-class performance on PQC algorithms.

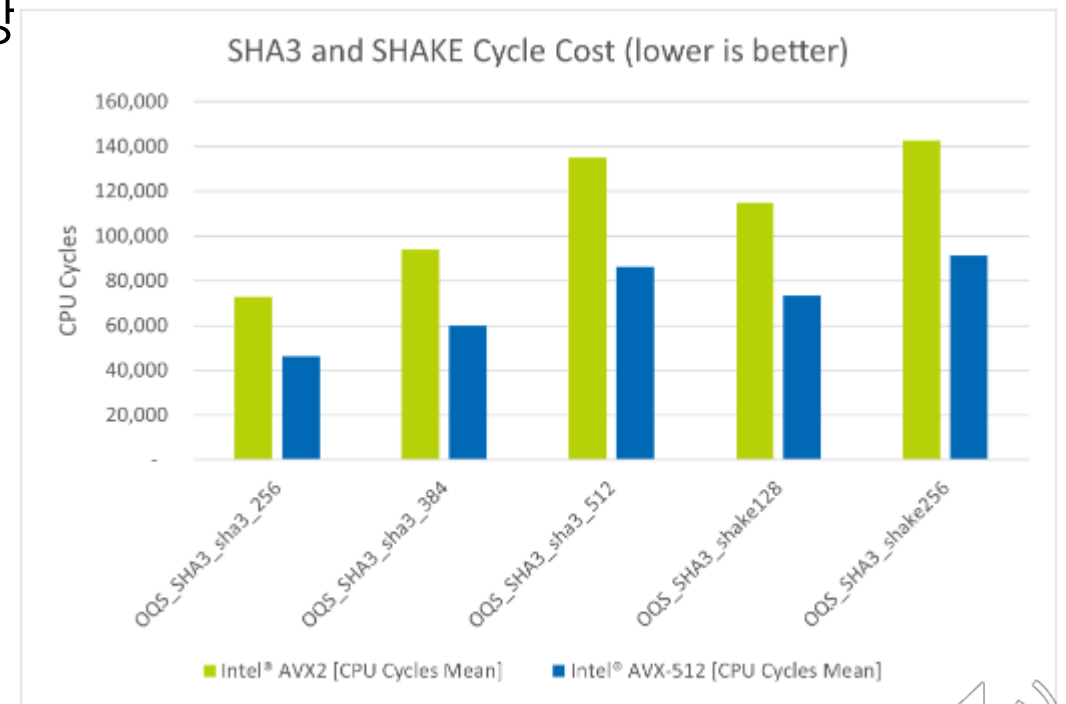


Figure 1. SHA3 and SHAKE Performance results with speed_common Tool on 8192 Byte Message (r2b/t/r)

<https://builders.intel.com/docs/networkbuilders/post-quantum-cryptography-accelerating-open-quantum-safe-library-with-intel-avx-512-keccak-1600-implementation-1762253806.pdf>

5. 최적화 구현 방법 - VPCLMULQDQ

- AImer: 영지식증명(MPCitH) 사용 → 파티(가상 참가자)가 GF 계산 수행 → 파티를 4개씩 묶어 성능 개선
→ 기존 AVX2는 2개씩 묶어 계산

AVX2

```
void GF_mul_add_N(GF c[AIMER_N], const GF a[AIMER_N], const GF b)
{
    __m128i x[2], y, z[2], zhi[2], t[4];
    __m128i irr = _mm_set_epi64x(0x0, 0x87);
    y = _mm_loadu_si128((const __m128i *)b);

    for (size_t party = 0; party < AIMER_N; party += 2)
    {
        x[0] = _mm_loadu_si128((const __m128i *)a[party]);
        x[1] = _mm_loadu_si128((const __m128i *)a[party + 1]);

        // polynomial multiplication x2
        t[0] = _mm_clmulepi64_si128(x[0], y, 0x01);
        t[1] = _mm_clmulepi64_si128(x[1], y, 0x01);
        t[2] = _mm_clmulepi64_si128(x[0], y, 0x10);
        t[3] = _mm_clmulepi64_si128(x[1], y, 0x10);

        z[0] = _mm_clmulepi64_si128(x[0], y, 0x00);
        z[1] = _mm_clmulepi64_si128(x[1], y, 0x00);
        zhi[0] = _mm_clmulepi64_si128(x[0], y, 0x11);
        zhi[1] = _mm_clmulepi64_si128(x[1], y, 0x11);

        t[0] = _mm_xor_si128(t[0], t[2]);
        t[1] = _mm_xor_si128(t[1], t[3]);

        z[0] = _mm_xor_si128(z[0], _mm_slli_si128(t[0], 8));
        z[1] = _mm_xor_si128(z[1], _mm_slli_si128(t[1], 8));
        zhi[0] = _mm_xor_si128(zhi[0], _mm_srli_si128(t[0], 8));
        zhi[1] = _mm_xor_si128(zhi[1], _mm_srli_si128(t[1], 8));

        // load c
        x[0] = _mm_loadu_si128((const __m128i *)c[party]);
        x[1] = _mm_loadu_si128((const __m128i *)c[party + 1]);

        // modular reduction x2
        t[0] = _mm_clmulepi64_si128(zhi[0], irr, 0x01);
        t[1] = _mm_clmulepi64_si128(zhi[1], irr, 0x01);

        z[0] = _mm_xor_si128(z[0], _mm_slli_si128(t[0], 8));
        z[1] = _mm_xor_si128(z[1], _mm_slli_si128(t[1], 8));
        zhi[0] = _mm_xor_si128(zhi[0], _mm_srli_si128(t[0], 8));
        zhi[1] = _mm_xor_si128(zhi[1], _mm_srli_si128(t[1], 8));

        t[0] = _mm_clmulepi64_si128(zhi[0], irr, 0x00);
        t[1] = _mm_clmulepi64_si128(zhi[1], irr, 0x00);

        z[0] = _mm_xor_si128(z[0], x[0]);
        z[1] = _mm_xor_si128(z[1], x[1]);

        z[0] = _mm_xor_si128(z[0], t[0]);
        z[1] = _mm_xor_si128(z[1], t[1]);

        _mm_storeu_si128((__m128i *)c[party], z[0]);
        _mm_storeu_si128((__m128i *)c[party + 1], z[1]);
    }
}
```

AVX512

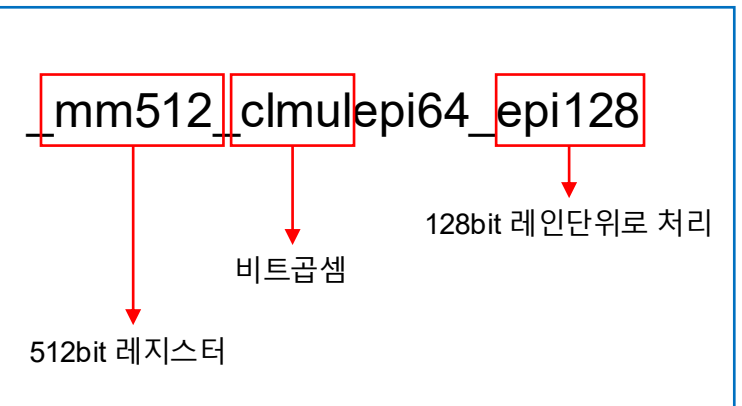
```
// c[i] ^= a[i] * b (4 parties per 512-bit register)
void GF_mul_add_N(GF c[AIMER_N], const GF a[AIMER_N], const GF b)
{
    const __m512i irr = _mm512_broadcast_i32x4(_mm_set_epi64x(0x0, 0x87));
    const __m512i y = _mm512_broadcast_i32x4(_mm_loadu_si128((const __m128i *)b));

    for (size_t party = 0; party < AIMER_N; party += 4)
    {
        __m512i x = _mm512_loadu_si512((const void *)a[party]);

        __m512i z = _mm512_clmulepi64_epi128(x, y, 0x00);
        __m512i zhi = _mm512_clmulepi64_epi128(x, y, 0x11);
        __m512i t = _mm512_xor_si512(_mm512_clmulepi64_epi128(x, y, 0x01),
                                     _mm512_clmulepi64_epi128(x, y, 0x10));
        z = _mm512_xor_si512(z, _mm512_bslli_epi128(t, 8));
        zhi = _mm512_xor_si512(zhi, _mm512_bslli_epi128(t, 8));

        // modular reduction (4-way)
        t = _mm512_clmulepi64_epi128(zhi, irr, 0x01);
        z = _mm512_xor_si512(z, _mm512_bslli_epi128(t, 8));
        zhi = _mm512_xor_si512(zhi, _mm512_bslli_epi128(t, 8));
        t = _mm512_clmulepi64_epi128(zhi, irr, 0x00);
        z = _mm512_xor_si512(z, t);

        // accumulate into c
        z = _mm512_xor_si512(z, _mm512_loadu_si512((const void *)c[party]));
        _mm512_storeu_si512((void *)c[party], z);
    }
}
```



어셈블리로 변환 시 VPCLMULQDQ로 출력 됨

왜 빨라질까요?

- 처리할 명령어 개수가 줄어들어서 빨라짐(코드 양이 줄어듦)



최적화 구현 방법 - VPTERNLOGQ

→ 행렬x벡터 함수의 AND + XOR 로직을 VPTERNLOGQ명령어 1개로 처리

```
void GF_transposed_matmul_add_N(GF c[AIMER_N], const GF a[AIMER_N],
                                const GF b[AIM2_NUM_BITS_FIELD])
{
    const __m256i zero = _mm256_setzero_si256();
    __m256i m0, m1, m2, m3, t;

    for (size_t party = 0; party < AIMER_N; party += 4)
    {
        const __m256i shift = _mm256_set_epi64x(0, 0, 1, 1);
        __m256i a0, a1, a2, a3, c0, c1, c2, c3;

        m0 = _mm256_loadu_si256((const __m256i *)a[party]);
        m1 = _mm256_loadu_si256((const __m256i *)a[party + 2]);
        m2 = _mm256_loadu_si256((const __m256i *)c[party]);
        m3 = _mm256_loadu_si256((const __m256i *)c[party + 2]);

        // a1 = a[party + 1].hi x4
        a0 = _mm256_permute4x64_epi64(m0, 0x55);
        a1 = _mm256_permute4x64_epi64(m0, 0xff);
        a2 = _mm256_permute4x64_epi64(m1, 0x55);
        a3 = _mm256_permute4x64_epi64(m1, 0xff);

        a0 = _mm256_sllv_epi64(a0, shift);
        a1 = _mm256_sllv_epi64(a1, shift);
        a2 = _mm256_sllv_epi64(a2, shift);
        a3 = _mm256_sllv_epi64(a3, shift);

        // c1 = [c[party + 1].lo | c[party + 1].hi | 0 | 0]
        c0 = _mm256_permute2x128_si256(m2, m2, 0x80);
        c1 = _mm256_permute2x128_si256(m2, m2, 0x81);
        c2 = _mm256_permute2x128_si256(m3, m3, 0x80);
        c3 = _mm256_permute2x128_si256(m3, m3, 0x81);

        for (size_t row = 128; row > 64; row -= 2)
        {
            // m1[w] = (a1[w].msb) ^ 64
            m0 = _mm256_cmpgt_epi64(zero, a0);
            m1 = _mm256_cmpgt_epi64(zero, a1);
            m2 = _mm256_cmpgt_epi64(zero, a2);
            m3 = _mm256_cmpgt_epi64(zero, a3);

            a0 = _mm256_slli_epi64(a0, 2);
            a1 = _mm256_slli_epi64(a1, 2);
            a2 = _mm256_slli_epi64(a2, 2);
            a3 = _mm256_slli_epi64(a3, 2);

            m0 = _mm256_and_si256(m0, t);
            m1 = _mm256_and_si256(m1, t);
            m2 = _mm256_and_si256(m2, t);
            m3 = _mm256_and_si256(m3, t);

            c0 = _mm256_xor_si256(c0, m0);
            c1 = _mm256_xor_si256(c1, m1);
            c2 = _mm256_xor_si256(c2, m2);
            c3 = _mm256_xor_si256(c3, m3);

            m0 = _mm256_loadu_si256((const __m256i *)a[party]);
            m1 = _mm256_loadu_si256((const __m256i *)a[party + 2]);

            // a1 = a[party + 1].lo x4
            a0 = _mm256_permute4x64_epi64(m0, 0x00);
            a1 = _mm256_permute4x64_epi64(m0, 0xaa);
            a2 = _mm256_permute4x64_epi64(m1, 0x00);
            a3 = _mm256_permute4x64_epi64(m1, 0xaa);

            a0 = _mm256_sllv_epi64(a0, shift);
            a1 = _mm256_sllv_epi64(a1, shift);
            a2 = _mm256_sllv_epi64(a2, shift);
            a3 = _mm256_sllv_epi64(a3, shift);

            m0 = _mm256_and_si256(m0, t);
            m1 = _mm256_and_si256(m1, t);
            m2 = _mm256_and_si256(m2, t);
            m3 = _mm256_and_si256(m3, t);

            c0 = _mm256_xor_si256(c0, m0);
            c1 = _mm256_xor_si256(c1, m1);
            c2 = _mm256_xor_si256(c2, m2);
            c3 = _mm256_xor_si256(c3, m3);

            a0 = _mm256_permute2x128_si256(c0, c1, 0x20);
            a1 = _mm256_permute2x128_si256(c0, c1, 0x31);
            a2 = _mm256_permute2x128_si256(c2, c3, 0x20);
            a3 = _mm256_permute2x128_si256(c2, c3, 0x31);

            c0 = _mm256_xor_si256(a0, a1);
            c2 = _mm256_xor_si256(a2, a3);

            _mm256_storeu_si256((__m256i *)c[party], c0);
            _mm256_storeu_si256((__m256i *)c[party + 2], c2);
        }
    }
}
```

```
void GF_transposed_matmul_add_N(GF c[AIMER_N], const GF a[AIMER_N],
                                const GF b[AIM2_NUM_BITS_FIELD])
{
    for (size_t p = 0; p < AIMER_N; p += 16)
    {
        __m512i v0 = _mm512_loadu_si512((const void *)a[p]);
        __m512i v1 = _mm512_loadu_si512((const void *)a[p + 4]);
        __m512i v2 = _mm512_loadu_si512((const void *)a[p + 8]);
        __m512i v3 = _mm512_loadu_si512((const void *)a[p + 12]);
        __m512i k0 = _mm512_loadu_si512((const void *)c[p]);
        __m512i k1 = _mm512_loadu_si512((const void *)c[p + 4]);
        __m512i k2 = _mm512_loadu_si512((const void *)c[p + 8]);
        __m512i k3 = _mm512_loadu_si512((const void *)c[p + 12]);
        __m512i h0 = _mm512_shuffle_epi32(v0, (_MM_PERM_ENUM)0xEE), l0 = _mm512_shuffle_epi32(v0, (_MM_PERM_ENUM)0x44);
        __m512i h1 = _mm512_shuffle_epi32(v1, (_MM_PERM_ENUM)0xEE), l1 = _mm512_shuffle_epi32(v1, (_MM_PERM_ENUM)0x44);
        __m512i h2 = _mm512_shuffle_epi32(v2, (_MM_PERM_ENUM)0xEE), l2 = _mm512_shuffle_epi32(v2, (_MM_PERM_ENUM)0x44);
        __m512i h3 = _mm512_shuffle_epi32(v3, (_MM_PERM_ENUM)0xEE), l3 = _mm512_shuffle_epi32(v3, (_MM_PERM_ENUM)0x44);

        for (int kk = 63; kk >= 0; kk--) {
            __m512i bc = _mm512_broadcast_i32x4(_mm_loadu_si128((const __m128i *)b[64 + kk]));
            k0 = _mm512_ternarylogic_epi64(k0, _mm512_srai_epi64(h0, 63), bc, 0x78);
            k1 = _mm512_ternarylogic_epi64(k1, _mm512_srai_epi64(h1, 63), bc, 0x78);
            k2 = _mm512_ternarylogic_epi64(k2, _mm512_srai_epi64(h2, 63), bc, 0x78);
            k3 = _mm512_ternarylogic_epi64(k3, _mm512_srai_epi64(h3, 63), bc, 0x78);
            h0 = _mm512_slli_epi64(h0, 1); h1 = _mm512_slli_epi64(h1, 1);
            h2 = _mm512_slli_epi64(h2, 1); h3 = _mm512_slli_epi64(h3, 1);
        }

        for (int kk = 63; kk >= 0; kk--) {
            __m512i bc = _mm512_broadcast_i32x4(_mm_loadu_si128((const __m128i *)b[kk]));
            k0 = _mm512_ternarylogic_epi64(k0, _mm512_srai_epi64(l0, 63), bc, 0x78);
            k1 = _mm512_ternarylogic_epi64(k1, _mm512_srai_epi64(l1, 63), bc, 0x78);
            k2 = _mm512_ternarylogic_epi64(k2, _mm512_srai_epi64(l2, 63), bc, 0x78);
            k3 = _mm512_ternarylogic_epi64(k3, _mm512_srai_epi64(l3, 63), bc, 0x78);
            l0 = _mm512_slli_epi64(l0, 1); l1 = _mm512_slli_epi64(l1, 1);
            l2 = _mm512_slli_epi64(l2, 1); l3 = _mm512_slli_epi64(l3, 1);
        }

        __mm512_storeu_si512((void *)c[p], k0);
        __mm512_storeu_si512((void *)c[p + 4], k1);
        __mm512_storeu_si512((void *)c[p + 8], k2);
        __mm512_storeu_si512((void *)c[p + 12], k3);
    }
}
```

BEFORE: 마스크 → [AND] → [XOR] (조건부 XOR = 명령 2개)

AFTER: 마스크 → [ternarylogic] (조건부 XOR = 명령 1개)



최적화 구현 방법 – Keccak(YMM기반 최적화)

- AVX-512VL(Vector Length Extensions)기반 Keccak permutation 최적화
- AVX-512VL은 YMM(256bit)레지스터를 사용하면서도 AVX-512명령어 사용 가능
 - YMM을 쓴다 ≠ AVX2 코드다.

AVX-512 전용 명령어	의미
kmovq	일반 레지스터의 비트마스크 값을 mask레지스터로 이동
vmovdqu8 {%k}	Mask를 사용해 필요한 byte만 load/store
vpternlogq	3개의 값을 원하는 bit연산으로 한번에 처리
vprolq	64bit lane 단위 rotate-left



최적화 구현 방법 – Keccak

- **vpternlogq**: 논리연산 3개를 합친 명령어
 - 64bit 값 3개를 넣고, 원하는 bit연산을 한 번에 수행하는 명령어

➤ $\text{tmp} = a \wedge b$
 $\text{tmp} = \text{tmp} \wedge c \longrightarrow \text{tmp} = a \wedge b \wedge c$

가능한 연산

$a \wedge b \wedge c$
 $a \& b$
 $a | b$
 $a \wedge ((\sim b) \& c)$
 $(a \& b) | (\sim a \& c)$

- **vprolq**: Rho 회전을 1번의 명령어로 처리
 - 64-bit 값을 왼쪽으로 rotate하는 명령어

- **AVX2**

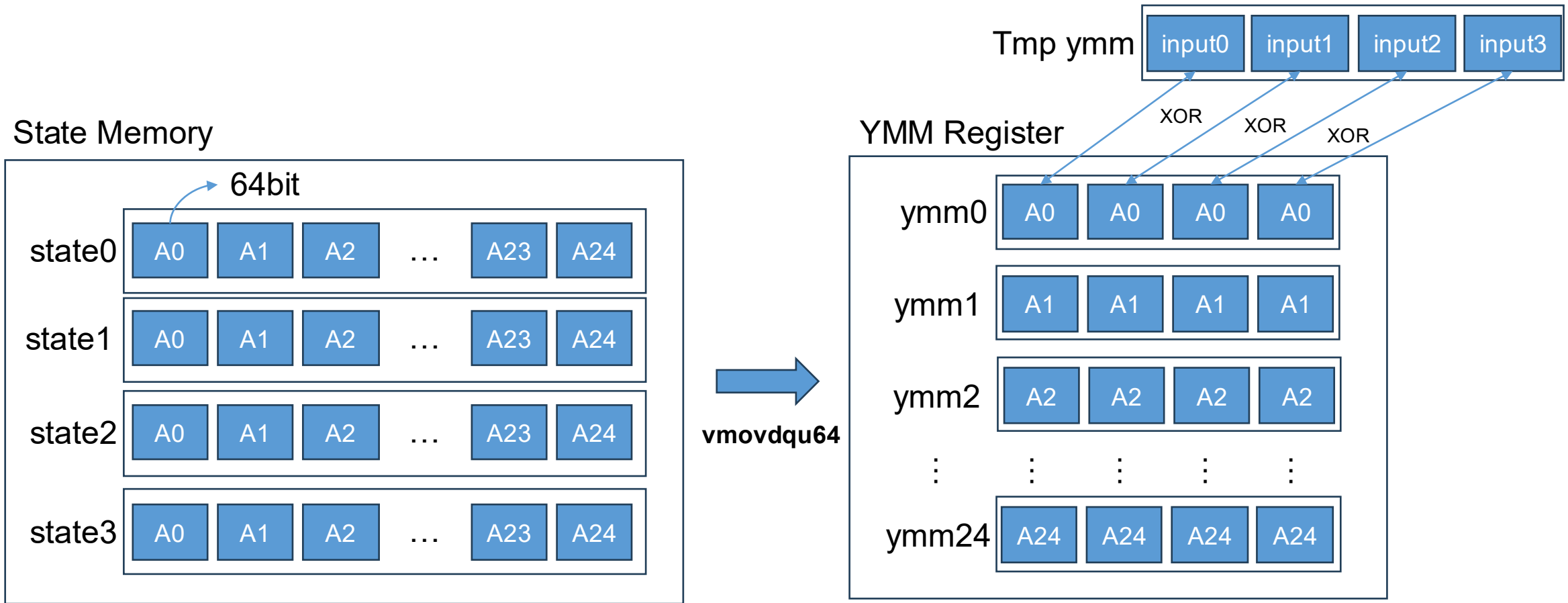
➤ $\text{left} = x \ll r$
 $\text{right} = x \gg (64-r)$
 $\text{rot} = \text{left} | \text{right}$

- **AVX512**

- `vprolq $44, %ymm6, %ymm6` → 명령어 1개로 해결
- **AVX2의 shift-left + shift-right + OR 조합을 대체한다.**



최적화 구현 방법 – Keccak state 병렬배치 후 XOR

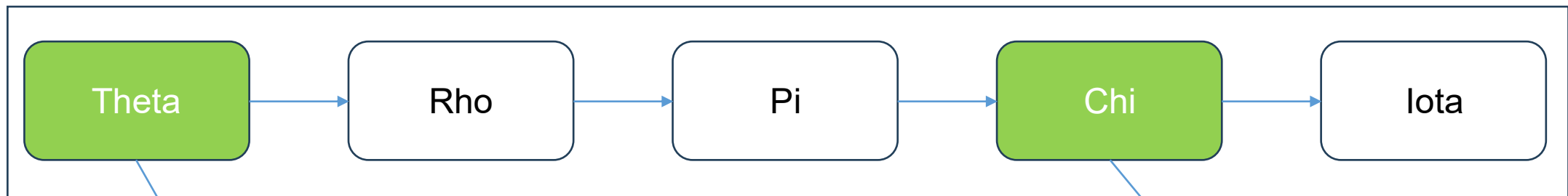


- 256bit YMM레지스터에 4개의 Keccak state를 병렬 배치
- 하나의 명령으로 4개의 Keccak state에 같은 연산을 동시에 수행 가능
 - YMM으로 옮긴 후 각 데이터(A0)를 원본 메시지 데이터(input)와 XOR연산(병렬진행)



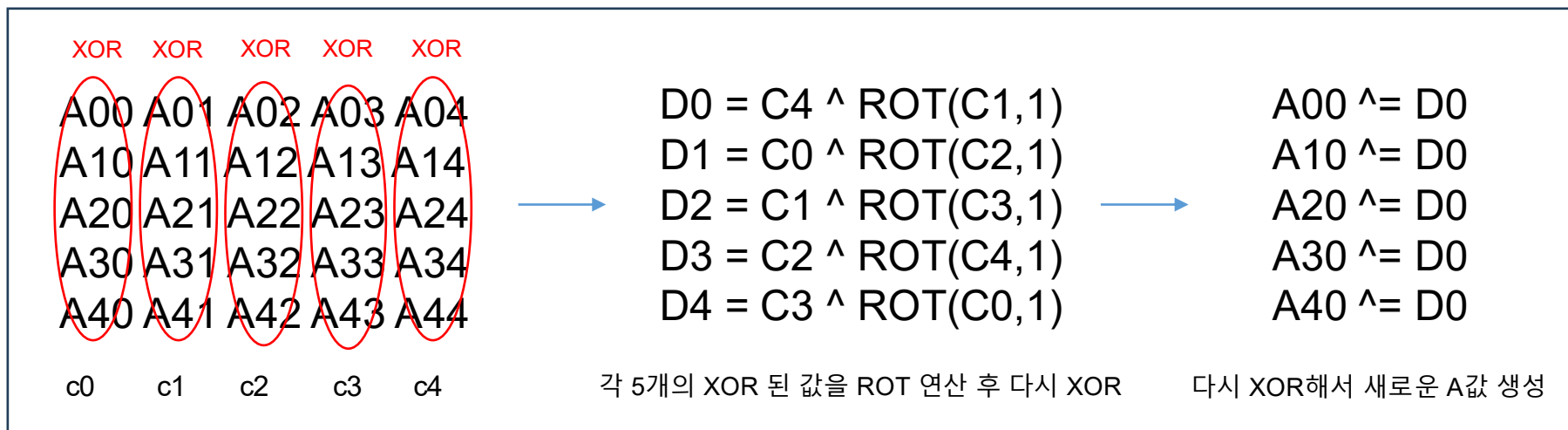
최적화 구현 방법 – Keccak Permutation

Keccak Permutation



전체 state 값을 섞어 diffusion 시키는 과정

NOT, AND, XOR 연산 부분에도 적용



3번의 XOR과 1번의 ROT 필요 → vpternlogq와 vprolq로 해결



6. AVX512 성능결과

항목	값
CPU	11th Gen Intel Core i7-1165G7 @ 2.80GHz (Tiger Lake)
코어	8 논리 스레드
주파수 정책	governor = performance , 터보 = on (no_turbo=0)
OS / 컴파일러	Ubuntu 24.04, gcc 13.3.0
최적화 플래그	-O3 -fomit-frame-pointer

서명(sign) — median cycles-50회 반복 측정

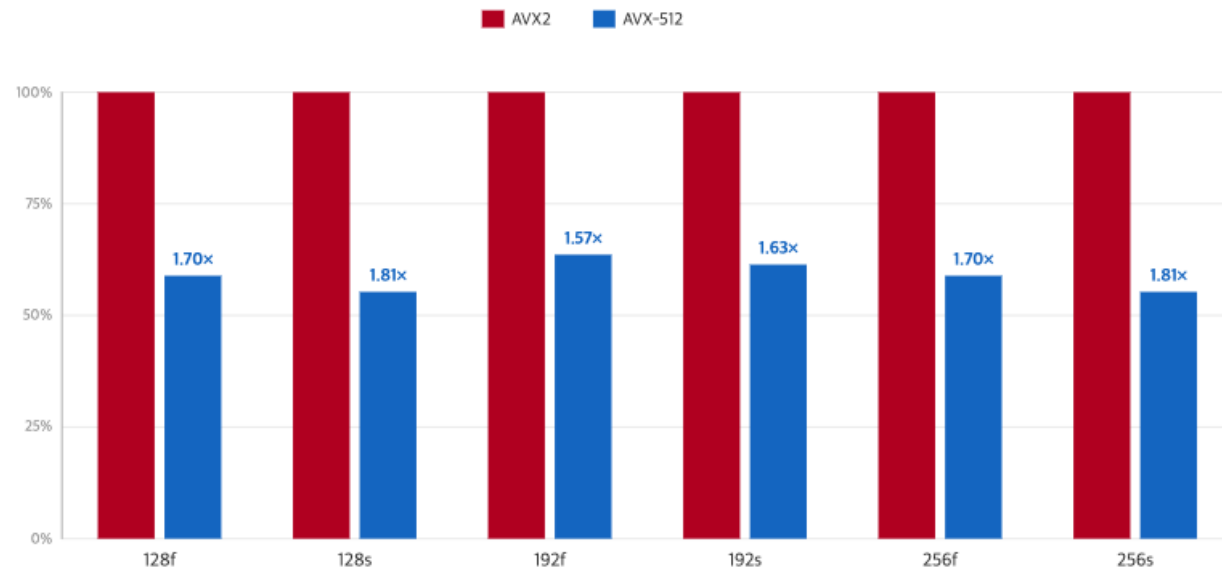
변종	avx2	*avx512	avx512/avx2
128f	802,112	471,423	1.70x
128s	5,996,114	3,308,898	1.81x
192f	2,025,983	1,287,691	1.57x
192s	15,072,915	9,254,646	1.63x
256f	4,003,931	2,355,574	1.70x
256s	28,686,277	15,852,569	1.81x

검증(verify) — 전체, median cycles-50회 반복 측정

변종	avx2	*avx512	avx512/avx2
128f	793,005	478,969	1.66x
128s	5,950,834	3,248,313	1.83x
192f	2,029,277	1,309,777	1.55x
192s	14,965,880	9,070,062	1.65x
256f	3,995,008	2,372,853	1.69x
256s	28,438,781	15,704,612	1.81x

AVX-512 vs AVX2 — 서명 속도 비교

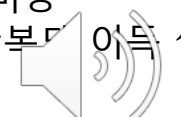
AVX2 = 100% 기준, 막대가 낮을수록 빠름 · 막대 위 숫자 = AVX-512 배속



AVX-512가 AVX2보다 평균 약 1.7배 빠름 (128·256 ≈ 1.8배, 192 ≈ 1.6배)



- 128 / 256 (≈1.7~1.8배): 숫자가 칸에 잘 맞아 "4명 동시"가 효율적
- 192 (≈1.6배, 제일 작음): 1.5칸이라 0 패딩 줄 세우기 부대비용
- s 변종 > f 변종: 파티가 256명이라 "4명 동시"가 더 많이 반복



7. 결론

① 4개씩 묶기 — 512비트 레지스터 하나에 4명분을 담아 한 명령으로

- field: 파티 4명의 GF 곱셈을 **vpclmulqdq**로 동시
- Keccak: 해시 4개를 **times4**로 동시

② **vpternlogq** — "입력 3개 논리연산"을 한 명령으로 융합

- field 행렬곱: 결과 $\wedge$ (마스크 & 행)
- Keccak χ 단계: $a \wedge (\sim b \& c)$ (60회!)

➤ "같은 일을 여러번 반복"하는 곳이 SIMD의 황금자리~



Q & A

